

色板游戏

色板长度为 L ， L 是一个正整数，所以我们可以均匀地将它划分成 L 块1厘米长的小方格。并从左到右标记为 $1, 2, \dots L$ 。

现在色板上只有一个颜色，老师告诉阿宝在色板上只能做两件事：

1. "C A B C" 指在 A 到 B 号方格中涂上颜色 C 。
2. "P A B" 指老师的提问： A 到 B 号方格中有几种颜色。

学校的颜料盒中一共有 T 种颜料。为简便起见，我们把他们标记为 $1, 2, \dots T$ 。开始时色板上原有的颜色就为1号色。面对如此复杂的问题，阿宝向你求助，你能帮助他吗？

输入格式

第一行有3个整数 L ($1 \leq L \leq 100000$), T ($1 \leq T \leq 30$) 和 O ($1 \leq O \leq 100000$)。在这里 O 表示事件数。

接下来 O 行，每行以 "C A B C" 或 "P A B" 得形式表示所要做的事情(这里 A, B, C 为整数, 可能 $A > B$ ，这样的话需要你交换 A 和 B)

色板游戏

我们发现如果只知道左右两个子区间的颜色种类，我们是没有办法知道父亲结点的颜色种类的，因为两个区间可能有相同的颜色。

看上去该题好像线段树不可以做，但是我们仔细看一下数据范围，颜色种类小于30，那么我们完全可以按位把每个位置的具体颜色存储下来，开一个int变量就可以了。

如果是区间修改，都涂上第 i 种颜色，那么我们还是只需要打上一个懒标记就可以了，我们可以先维护一个和，最后再看这个和里面有多少个1，所以区间修改就是 $sum = 1 \ll (i - 1)$ ， $lazy = 1$ ；合并左右两个子区间的时候用“或”运算就可以了，只要该位上有一个1，答案就是1。最后再用位运算取出所有的1就可以了。



WM 度假

题目描述

小 W 和小 M 来到国度假辣！

E 国素有万岛之国的名，共有 N 座岛屿，每座岛屿有真、善、美三个属性。对于一座岛屿，如果没有其它某岛屿三个属性都超过它，那么小 W 和小 M 会选择到这座岛屿度假。

小 W 和小 M 一共会去多少座岛屿度假呢？



WM 度假

该题是判断是否存在一个岛的三个属性都比某一个岛值小，那么就不会选该岛。

如果是只有两个属性那倒是可以直接做，先按照第一个属性从大到小排序，这样我们就保证了后面的岛屿的第一个属性比前面岛屿小，然后在对于每一个岛屿的第二个属性，看看前面是否存在第二个属性值比当前岛大的，如果有，那么说明该岛不选，如果没有，那么说明该岛要选。用线段树或者树状数组维护一下前缀最大值就可以了。

但是该题是三个属性，那么我们又该如何处理呢？

我们可以换一种思路：如果只有两个属性，但是我们不能改变他的序列，要么要如何判断是否存在一个岛各个属性都大于他。

WM 度假

对于有两种属性的题，我们可以把其中一个属性看做区间，其中一个属性看做权值，比如两个属性 b_i, c_i ，我们把 b_i 看做区间编号， c_i 看做权值，那么我们先把 b_i 这个位置修改为 c_i ，对于下一次枚举的数 b_j, c_j ，我们先去查询区间 $[1, b_j]$ ，如果区间 $[1, b_j]$ 中存在值，那么说明了

(1) 有编号在他前面的数——存在岛 $a[]$ 值比他小

(2) 该区间内有值——存在岛 $b[]$ 值比他小

那么如果还存在一个岛 $c[]$ 值也比他小，那么这个岛就不需要选择了，我们这压力统计不选的岛，剩下的就是需要选择的岛。

怎么判断 $c[]$ 值呢，我们维护一个最小值，如果区间 $[1, b_j]$ 的最小值比 c_j 小，那么就说存在，否则就更新 b_j 这个位置的最小值为 c_j

花神游历各国

"第一分钟，X说，要有数列，于是便给定了一个正整数数列。

第二分钟，L说，要能修改，于是便有了对一段数中每个数都开平方(下取整)的操作。

第三分钟，k说，要能查询，于是便有了求一段数的和的操作。

第四分钟，彩虹喵说，要是noip难度，于是便有了数据范围。

第五分钟，诗人说，要有韵律，于是便有了时间限制和内存限制。

第六分钟，和雪说，要省点事，于是便有了保证运算过程中及最终结果均不超过64位有符号整数类型的表示范围的限制。

第七分钟，这道题终于造完了，然而，造题的神牛们再也不想写这道题的程序了。"

花神游历各国

该题如果我们知道两个子区间的平方和，我们可以直接得到父亲区间的平方和。

但是，该题涉及区间修改，我们是没办法添加懒标记的，因为开方和和原数据之和没有太大关系。

那么该题要如何维护呢？

就需要一定的数学思维了。一个int范围内的数最多开多少次方就不能再开方了，一旦结果变为1再开方也就没意义了。

$$4*10^9 \rightarrow 6*10^4 \rightarrow 3*10^2 \rightarrow 17 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

所以我们发现一个数，最多开不会超过6次方就会变为1，那么我们该题直接暴力开方就好，一个点最多被访问6次就会变为1，如果一个区间都变为1，那么打上标记，后面就不需要开放了，时间复杂度还是 $n \log n$ 级别

收作业

题目描述

xlj是一名来自00001110班的妹子，她是英语科代表。

xlj回到了教室，可怜的她要收英语作业，但是00001110班这群不负责任的组长把作业收得乱七八糟，散得每个座位上都有作业本，xlj只好挨个去收。

00001110班的教室可以看作是一个 n 行 m 列的矩形，xlj在 $(0,0)$ 这个格子（位于教室的左下角），教室的门在 $(n-1,m-1)$ 这个格子。每次xlj可以向相邻的格子走一步，走到某个格子时，她会收完这个格子的英语作业。xlj还有许多事要做，她想走一条最短路到教室的门口，但是作业收得太少了又会引起公愤，所以她决定走一条作业收得最多的最短路线(即保证路径最短的前提下收的作业最多)，你能帮帮她吗？

xlj是一名来自00001110班的妹子，她是英语科代表。

xlj回到了教室，可怜的她要收英语作业，但是00001110班这群不负责任的组长把作业收得乱七八糟，散得每个座位上都有作业本，xlj只好挨个去收。

00001110班的教室可以看作是一个 n 行 m 列的矩形，xlj在 $(0,0)$ 这个格子（位于教室的左下角），教室的门在 $(n-1,m-1)$ 这个格子。每次xlj可以向相邻的格子走一步，走到某个格子时，她会收完这个格子的英语作业。xlj还有许多事要做，她想走一条最短路到教室的门口，但是作业收得太少了又会引起公愤，所以她决定走一条作业收得最多的最短路线(即保证路径最短的前提下收的作业最多)，你能帮帮她吗？



收作业

一道裸的线段树优化dp的题，DP思想就一个简单的去前面找最大值，用线段树来维护就可以了。



线段树

线段树可以用来求解大部分区间问题，他的用途很广泛，今天主要介绍一种最长串的问题。

有一个 n 个元素的数组，给出每个元素初始状态（不是0就是1）。有 m 条指令，指令有3种：

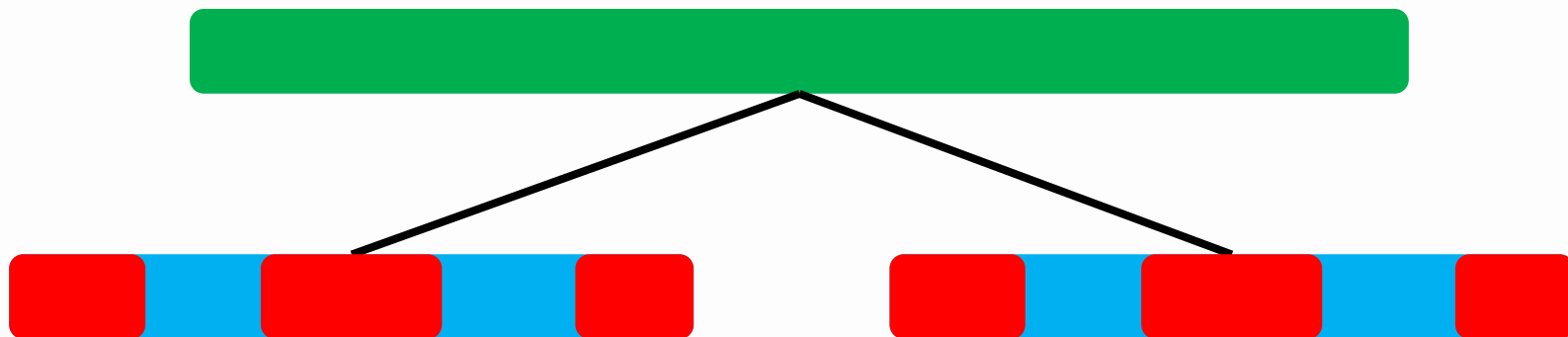
0. 将 $l \cdots r$ 的所有元素改为0
1. 将 $l \cdots r$ 的所有元素改为1
2. 统计 $l \cdots$ 中最长的连续1的长度

建树

(1) 建树

建树很容易，和普通线段树建树一样，当区间长度为1的时候，如果值为1，更新长度为1，值为0，更新长度为0。

但是，当求解父亲结点的区间长度的时候就需要变化了。观察下图。已知两个孩子结点的最长1的长度，其父亲结点的最长1的长度可以直接取max得到吗？



红色表示1，蓝色表示0

建树

并不是，可能就是其左右子区间的最长长度的最大值，但是也有可能不是，因为此时左右两个子区间构成了一个大区间，很有可能左区间的右边+右区间的左边组成的连续的1的数量更大，所以我们不能仅仅只维护区间最长1的长度，还需要维护区间左右两端最长1的长度。

```
struct node
{
    int l, r, lazy, len, lmax, rmax, zmax;
} tree[maxn*4];
```



红色表示1，蓝色表示0

建树

```
void updatafa(int id)
{
    if(tree[id*2].lmax==tree[id*2].len)//左子树全1
        tree[id].lmax=tree[id*2].len+tree[id*2+1].lmax;
    else
        tree[id].lmax=tree[id*2].lmax;
    if(tree[id*2+1].rmax==tree[id*2+1].len)//右子树全1
        tree[id].rmax=tree[id*2+1].len+tree[id*2].rmax;
    else
        tree[id].rmax=tree[id*2+1].rmax;
    tree[id].zmax=max(tree[id*2].zmax,tree[id*2+1].zmax);
    tree[id].zmax=max(tree[id].zmax,tree[id*2].rmax+tree[id*2+1].lmax);
}
```

建树

我们再来考虑父亲区间的左右总三个的长度。
我们会发现左右连续1的长度也有两种情况



父亲区间的左最长=左孩子区间的左最长



父亲区间的左最长=左孩子区间长度+右孩子区间左最长

父亲区间的总最长= $\max$ (左孩子区间总最长, 右孩子区间总最长, 左孩子区间右最长+右孩子区间左最长)

区间更新

更新也是在原来的线段树更新上修改：

如果区间变为0，那么这一段的所有max的值都变为0

如果区间变为1，那么这一段的所有max的值都变为len

如果需要访问到下层区间，判断是否有懒标记，如果有，先把懒标记下传。

子区间更新，其父亲区间也需要更新，更新方式和建树时的更新一模一样。所以可以把更新父亲区间封装成一个函数，这里直接调用就可以了

区间查询

区间查询就要稍微复杂一点，因为查询的区间在线段树上可能由好几部分组成，最终的总的最长可能和前面建树时讨论的情况一样，可能是组合的，所以返回值的时候要注意，不能直接返回最长的长度，而是应返回一个结构体，因为我们还需要当前区间的左右总最长。

查询结束之后，更新父亲区间，更新方式和前面建树时的更新比较类似，分别更新这个结点的左右总最长，并且把这个结构体返回回去。

区间查询

```
if(r<=mid)
return query(id*2,l,r);
else if(l>mid)
return query(id*2+1,l,r);
else
{
    node t1=query(id*2,l,mid);
    node t2=query(id*2+1,mid+1,r);
    node tmp;
    if(t1.lmax==t1.len)
    tmp.lmax=t1.len+t2.lmax;
    else
    tmp.lmax=t1.lmax;
    if(t2.rmax==t2.len)
    tmp.rmax=t2.len+t1.rmax;
    else
    tmp.rmax=t2.rmax;
    tmp.zmax=max(t1.zmax,t2.zmax);
    tmp.zmax=max(tmp.zmax,t1.rmax+t2.lmax);
    return tmp;
}
```



最长连续上升子序列

题目描述

给定 n 个整数，你有两个操作：

U A B：将编号为 A 的位置的数替换为数字 B (编号从 0 开始)

Q A B：输出区间 $[a,b]$ 中最长严格连续上升子序列的长度,保证区间合理





解题思路

该题和上一道题几乎一样，仍然是维护一个左最长，右最长，总最长，因为要判断两个子区间合并时是否会更新长度，所以还需要维护区间端点的值。



旅馆

题目描述

奶牛们最近的旅游计划，是到苏必利尔湖畔，享受那里的湖光山色，以及明媚的阳光。作为整个旅游的策划者和负责人，贝茜选择在湖边的一家著名的旅馆住宿。这个巨大的旅馆一共有 N ($1 \leq N \leq 50,000$)间客房，它们在同一层楼中顺次一字排开，在任何一个房间里，只需要拉开窗帘，就能见到波光粼粼的湖面。贝茜一行，以及其他慕名而来的旅游者，都是一批批地来到旅馆的服务台，希望能订到 D_i ($1 \leq D_i \leq N$)间连续的房间。服务台的接待工作也很简单：如果存在 r 满足编号为 $r..r+D_i-1$ 的房间均空着，他就将这一批顾客安排到这些房间入住；如果没有满足条件的 r ，他会道歉说没有足够的空房间，请顾客们另找一家宾馆。如果有多个满足条件的 r ，服务员会选择其中最小的一个。旅馆中的退房服务也是批量进行的。每一个退房请求由2个数字 X_i 、 D_i 描述，表示编号为 $X_i..X_i+D_i-1$ ($1 \leq X_i \leq N-D_i+1$)房间中的客人全部离开。退房前，请求退掉的房间中的一些，甚至是所有，可能本来就无人入住。而你的工作就是写一个程序，帮服务员为旅客安排房间。你的程序一共需要处理 M ($1 \leq M \leq 50,000$)个按输入次序到来的住店或退房的请求。第一个请求到来前，旅店中所有房间都是空闲的。

解题思路

该题其实和前面的最长串的题目大部分操作都是一样的，唯一不一样的地方在于修改操作如果有多段满足条件的区间要找编号最小的，即最左边的。

所以，如果左子树空余区间长度 $\geq k$ ，那么直接去左边找。如果是左子树的右最长+右子树的左最长 $\geq k$ ，那么就没必要去找了，直接返回该区间的左端点就可以了，否则就去右子树查询。

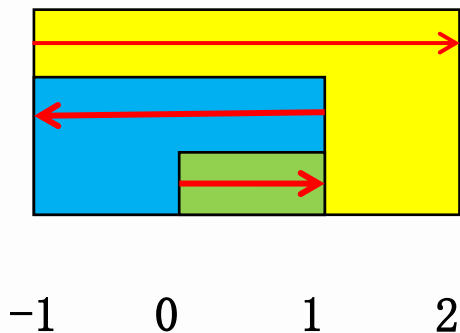
解题思路

```
int query(int id,int len)
{
    if(tree[id].lazy) //标记下传
        pushdown(id);
    if(tree[id*2].zmax>=len)//到左区间去查找
    {
        return query(id*2,len);
    }
    else if(tree[id*2].rmax+tree[id*2+1].lmax>=len)//一定是左区间最后一个端点-左区间右最长+1
    {
        return tree[id*2].r-tree[id*2].rmax+1;//这个地方前面没有pushdown会错。
        //因为你需要用到tree[id*2].rmax, 如果不pushdown, 这个值是没有更新的
    }
    else
    {
        return query(id*2+1,len);
    }
}
```

//因为需要编号最小的, 所以左子树有就去左子树, 如果在中间, 实际上可以直接得到, 不需要递归
//如果右子树有, 就去右子树

扫描线

每一次操作都会有一个区间，我们把每个区间看做一个矩形，那么涂色的次数就是矩形重叠的区域。



输入:

3 2

1 R

2 L

3 R

1、存储矩形竖着的每一条边，由于分左边界(开始)和右边界(结束)，所以我们分别用1和-1来表示，进入矩形，次数+1，离开矩形，次数-1，这样当我遍历完这个矩形的时候，又恢复成0了。

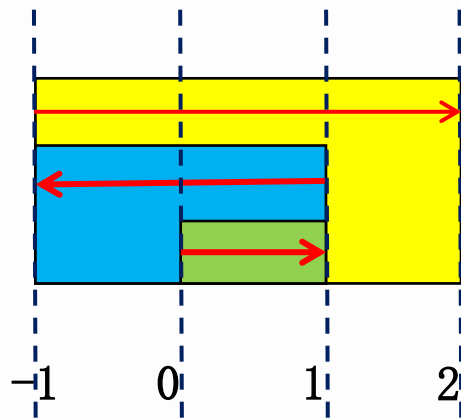
扫描线

2、把线段从左往右排序(根据题目需要)。

(位置, 起点还是终点)

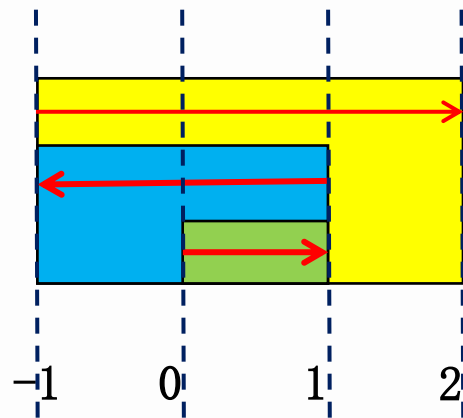
$(-1, 1)$, $(-1, 1)$, $(0, 1)$, $(1, -1)$, $(1, -1)$, $(2, -1)$

3、构造一条竖着的扫描线, 从左往右移动, 移动到第 i 条边的时候, 加上当前这条边的第二个参数。如果 $\geq k$, 那么 i 和 $i-1$ 条边的距离只差就是满足条件的区间。



扫描线

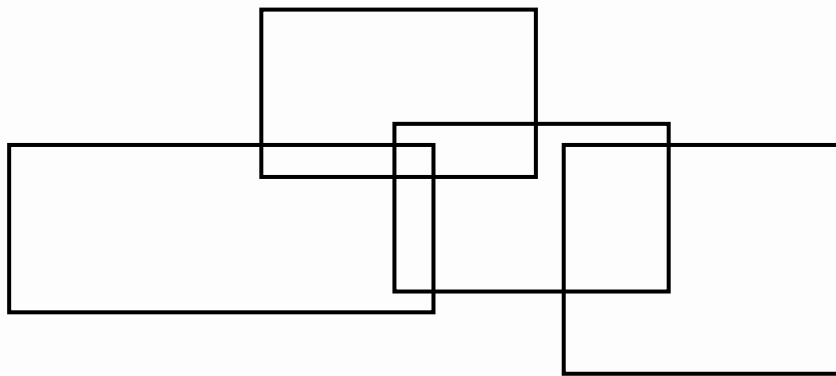
- (1) 初始时 $\text{cnt}=0$ 遍历第一条边 $(-1, 1)$ $\text{cnt}+1$
(2) 此时 $\text{cnt}=1$ 遍历第二条边 $(-1, 1)$ $\text{cnt}+1$
(3) 此时 $\text{cnt}=2 \geq k$ 遍历第三条边 $(0, 1)$, 得到满足条件的区间 $\text{ans}=0-(-1)=1$ $\text{cnt}+1$
(4) 此时 $\text{cnt}=3 \geq k$ $\text{ans}=2$ $\text{cnt}-1$
(5) 此时 $\text{cnt}=2 \geq k$ $\text{ans}=2+0$ $\text{cnt}-1$
(6) 此时 $\text{cnt}=1$ $\text{cnt}-1$
所以最终 $\text{ans}=2$;



```
now=line[1].val;  
for(i=2;i<=1;++i)  
{  
    if(now>=m) ans+=line[i].x-line[i-1].x;  
    now+=line[i].val;  
}
```

矩形面积之和

题目大意：给你 n 个矩形，求这些矩形面积之和
(矩形之间可能相交)。 $N \leq 100000$; 坐标 $\leq 1e9$



矩形面积之和

这是一道离散化+扫描线+线段树的经典题目。

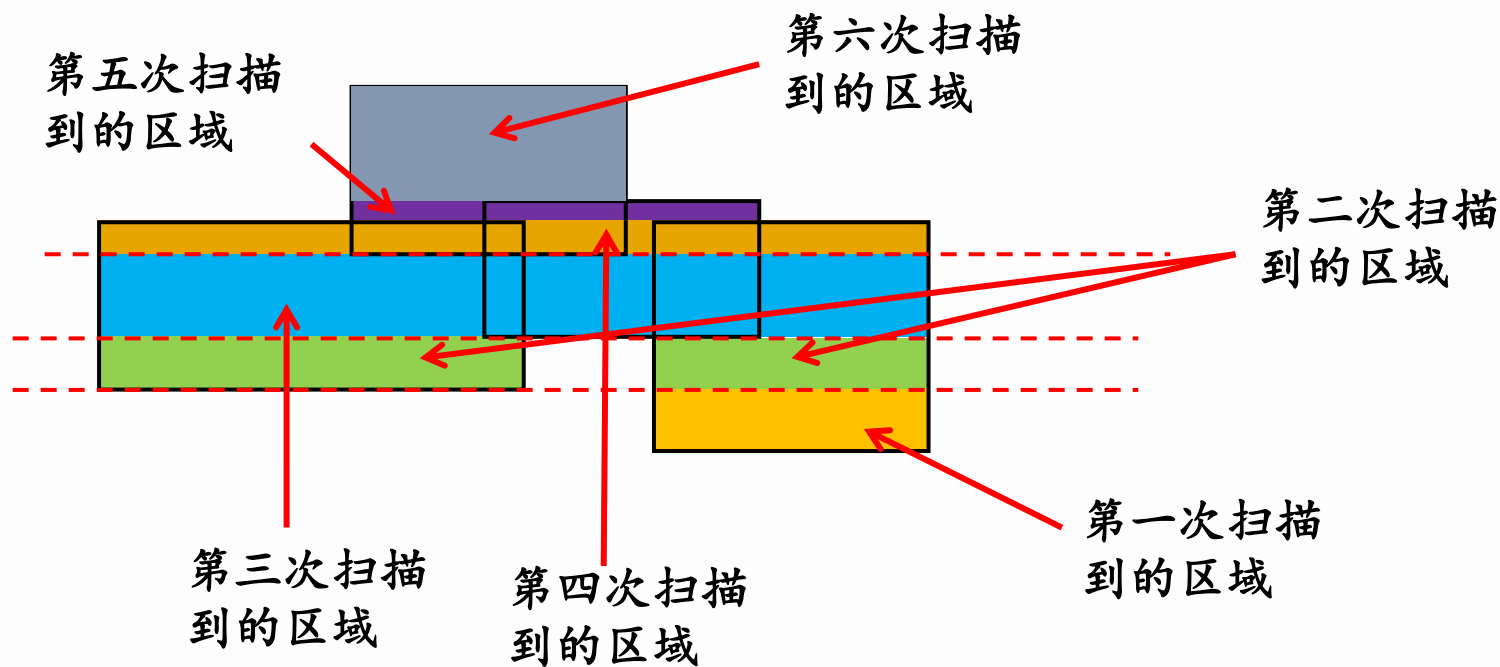
1、离散化

由于坐标特别大，空间会炸掉，所以我们需要离散化，如果我们的扫描线是横向的，那么我们把横坐标离散化，如果是纵向的，把纵坐标离散化

2、保存边

存储这条边，由于是平行线，所以只需要存储左右点的x坐标，和一个y坐标就可以了，再用1和-1记录是起点边还是终点边。

矩形面积之和



3、把边按照y值从小到大排序，然后从下到上扫描上下边，扫描到下边时，区间值+1，扫描到上边时，区间值-1，，用（下一条边-上一条边）*此时区间有效长度（非0的个数）来计算面积。

实现过程

第一步：先重新开两个数组来记录，一个数组记录横着的每一条线 $(y, x1, x2)$ ，以及这条线是矩形区域的进入边还是出去边，进入边，表示长度增加，用1表示，出去边表示已有的长度会较少，用-1表示，另一个数组记录则每一个 x 的坐标。然后再从小到大排序

```
for(int i=1;i<=n;i++)
{
    scanf("%lf%lf%lf%lf",&x1,&y1,&x2,&y2);
    k++;
    p[k].y=y1;p[k].x1=x1;p[k].x2=x2;p[k].flag=1;//进入边
    x[k]=x1;
    k++;
    p[k].y=y2;p[k].x1=x1;p[k].x2=x2;p[k].flag=-1;
    x[k]=x2;
}
```

实现过程

第二步：建线段树，但是要注意，该题实际上是边权，而不是点权，比如 $[1, 2]$ 实际上表示长度为1的线段，不能再分了， $[1, 1]$ 根本不是表线段长度，并且，如果用 $[1, 2]$ 和 $[3, 4]$ 表示，那么很明显 $[2, 3]$ 长度丢失了. 并且线段树还需要记录区间实际左右端点位置。

```
tree[id].l=l;tree[id].r=r;
tree[id].fl=x[l];tree[id].fr=x[r];
tree[id].cf=tree[id].flen=0;//一定要有，因为多组数据
if(l+1==r)//因为是边权，所以线段树和普通线段树有点不一样
return ;
int mid=(l+r)/2;
build(id*2,l,mid);
build(id*2+1,mid,r);//如果是mid和mid+1,那么就少了一截
```

实现过程

```
void update(int id, point a)
{
    if(tree[id].fl==a.x1&&tree[id].fr==a.x2)
    {
        tree[id].cf+=a.flag;
        js(id); // 计算覆盖长度, 重复的只算一次
        return ;
    }
    if(a.x2<=tree[id*2].fr) // 如果该线段都在左区间
        update(id*2, a);
    else if(a.x1>=tree[id*2+1].fl) // 如果都在右区间
        update(id*2+1, a);
    else // 如果两个区间各一部分
    {
        point t1=a, t2=a;
        t1.x2=tree[id*2].fr;
        update(id*2, t1);
        t2.x1=tree[id*2+1].fl;
        update(id*2+1, t2);
    }
    js(id); // 一定不要忘记更新根节点的信息
}
```



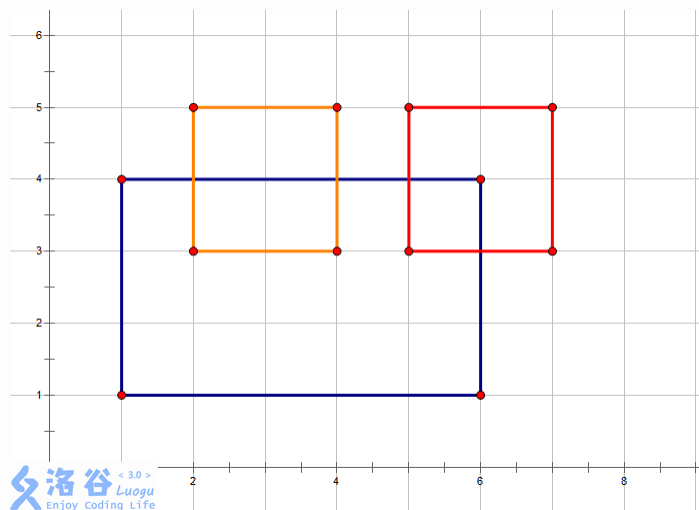
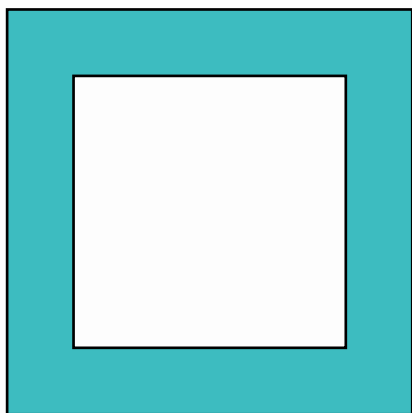
实现过程

第四步：查询答案，我们发现根本不需要写线段树的查询函数，因为每次都是求总的长度，所以直接找到根结点的覆盖长度就是答案，然后再乘以高度只差就是当前两条线段之间的面积。



矩形周长之和

上题问面积之和是多少，那么如果问你周长之和呢？周长也就是围成的形状的表面的边长之和（图中黑色线条为周长）。



矩形周长之和

首先，矩形的周长分为长和宽，如果我们按照前面求面积的方法来做，宽度就不太好求，所以我们可以把矩形的长和宽分开来求解，即先求水平的，再求垂直的。

在考虑的时候，我们发现要分上底面和下底面考虑。

对于下底面的边 $[s, t]$ ，先查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是新增的

对于上底面的边 $[s, t]$ ，先删除 $[s, t]$ 对应的边，然后再查询 $[s, t]$ 之间被覆盖的长度 x ，然后 $t-s-x$ 就是。



练习题

洛谷	1558	色板游戏
MSOJ	1470	WM度假
洛谷	4145	花神游历各国
MSOJ	1546	收作业
洛谷	2572	序列操作
MSOJ	1443	最长连续上升子序列
MSOJ	1462	旅馆
洛谷	1502	窗口的星星
洛谷	1856	矩形的周长

